



API 명세서

담당자	 [INT1]한민희
진행 상태	완료
프로젝트	주제 선정/기획
우선순위	낮음
태그	

API 명세서

본 문서는 아파트 관리 서비스 MVP에서 제공하는 **Spring Boot 기반 일반 서비스 API**와 **AWS Lambda 기반 RAG API**를 구분하여 정의한다.

일반 서비스는 **EC2 → Spring Boot → PostgreSQL** 구조로 동작하며, 관리규정 RAG는 별도의 **API Gateway → Lambda → OpenSearch / Bedrock / S3** 구조로 동작한다.

1. 공통 규약

1.1 Base Path

일반 서비스

```
/api/v1
```

RAG

```
/rag/v1
```

RAG는 별도 시스템이므로 일반 서비스 API와 독립적으로 관리한다.

1.2 인증

일반 서비스의 로그인 이후 JWT Access Token을 사용한다.

```
Authorization: Bearer <accessToken>
```

인증이 필요한 API는 JWT에서 다음 정보를 확인한다.

```
{
  "memberId": 1,
  "role": "USER",
  "unitId": 101,
  "apartmentId": 10
}
```

실제 JWT에는 필요한 최소 정보만 포함하고, 아파트/세대 정보는 필요에 따라 DB에서 재확인한다.

1.3 권한

권한	설명
USER	일반 입주민
ADMIN	아파트 관리자

관리자 API는 ADMIN 권한을 가진 사용자만 호출할 수 있다.

1.4 공통 응답 형식

성공

```
{
  "resultCode": "200-1",
  "msg": "요청이 성공했습니다.",
  "data": {}
}
```

실패

```
{
  "resultCode": "400-1",
  "msg": "잘못된 요청입니다.",
  "data": null
}
```

1.5 공통 에러 코드

코드	의미
400-1	잘못된 요청 / 입력값 검증 실패
401-1	인증 실패 / 로그인 필요
403-1	권한 부족
404-1	대상 리소스 없음

코드	의미
409-1	중복 / 충돌
500-1	서버 내부 오류

2. API 목록

영역	기능	Method	Endpoint	인증
인증	회원가입	POST	/api/v1/auth/signup	불필요
인증	로그인	POST	/api/v1/auth/login	불필요
인증	토큰 재발급	POST	/api/v1/auth/refresh	Refresh Token
인증	로그아웃	POST	/api/v1/auth/logout	필요
회원	내 정보 조회	GET	/api/v1/members/me	필요
아파트	내 아파트 조회	GET	/api/v1/apartments/me	필요
공지	공지 목록	GET	/api/v1/notices	필요
공지	공지 상세	GET	/api/v1/notices/{noticeId}	필요
공지	공지 등록	POST	/api/v1/admin/notices	ADMIN
공지	공지 수정	PATCH	/api/v1/admin/notices/{noticeId}	ADMIN
공지	공지 삭제	DELETE	/api/v1/admin/notices/{noticeId}	ADMIN
긴급	긴급 알림 목록	GET	/api/v1/emergency-alerts	필요
긴급	긴급 알림 상세	GET	/api/v1/emergency-alerts/{alertId}	필요
긴급	긴급 알림 등록	POST	/api/v1/admin/emergency-alerts	ADMIN
긴급	긴급 알림 종료	PATCH	/api/v1/admin/emergency-alerts/{alertId}/resolve	ADMIN
민원	민원 등록	POST	/api/v1/complaints	필요
민원	내 민원 조회	GET	/api/v1/complaints/me	필요
민원	민원 상세	GET	/api/v1/complaints/{complaintId}	필요
민원	관리자 민원 목록	GET	/api/v1/admin/complaints	ADMIN
민원	민원 상태 변경	PATCH	/api/v1/admin/complaints/{complaintId}/status	ADMIN
관리비	내 관리비 목록	GET	/api/v1/management-fees/me	필요
관리비	관리비 상세	GET	/api/v1/management-fees/{feeId}	필요
관리비	관리비 분석	GET	/api/v1/management-fees/{feeId}/analysis	필요
RAG	규정 질문	POST	/rag/v1/chat	필요
RAG	규정 문서 등록	POST	/rag/v1/admin/documents	ADMIN
RAG	규정 문서 목록	GET	/rag/v1/admin/documents	ADMIN
RAG	규정 문서 삭제	DELETE	/rag/v1/admin/documents/{documentId}	ADMIN

3. 인증 / 회원 API

A-01. 회원가입

POST /api/v1/auth/signup

Request

```
{
  "email": "user@example.com",
  "password": "password123!",
  "name": "홍길동",
  "apartmentId": 1,
  "buildingId": 3,
  "unitId": 1201
}
```

Response

```
{
  "resultCode": "200-1",
  "msg": "회원가입이 완료되었습니다.",
  "data": {
    "memberId": 1
  }
}
```

정책

- 이메일 중복 확인
- 비밀번호는 해시하여 저장
- 사용자의 세대 정보 확인
- **USER** 권한으로 가입

A-02. 로그인

POST /api/v1/auth/login

Request

```
{
  "email": "user@example.com",
  "password": "password123!"
}
```

Response

```
{
  "resultCode": "200-1",
  "msg": "로그인되었습니다.",
  "data": {
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "expiresIn": 3600
  }
}
```

A-03. Access Token 재발급

POST /api/v1/auth/refresh

Request

```
{
  "refreshToken": "eyJ..."
}
```

Response

```
{
  "resultCode": "200-1",
  "msg": "토큰이 재발급되었습니다.",
  "data": {
    "accessToken": "eyJ...",
    "refreshToken": "eyJ..."
  }
}
```

정책

- Refresh Token 검증
- 기존 Refresh Token 무효화
- 새로운 Access Token / Refresh Token 발급
- Refresh Token Rotation 적용

A-04. 로그아웃

POST /api/v1/auth/logout

인증

필요

Response

```
{
  "resultCode": "200-1",
  "msg": "로그아웃되었습니다.",
  "data": null
}
```

4. 회원 API

B-01. 내 정보 조회

GET /api/v1/members/me

Response

```
{
  "resultCode": "200-1",
  "msg": "회원 정보를 조회했습니다.",
  "data": {
    "memberId": 1,
    "name": "홍길동",
    "email": "user@example.com",
    "role": "USER",
    "apartment": {
      "id": 1,
      "name": "00아파트"
    },
    "building": {
      "id": 3,
      "buildingNumber": "103"
    },
    "unit": {
      "id": 1201,
      "unitNumber": "1201"
    }
  }
}
```

5. 아파트 API

C-01. 내 아파트 정보 조회

```
GET /api/v1/apartments/me
```

Response

```
{
  "resultCode": "200-1",
  "msg": "아파트 정보를 조회했습니다.",
  "data": {
    "apartmentId": 1,
    "name": "00아파트",
    "address": "서울특별시 ..."
  }
}
```

정책

JWT의 사용자 정보를 기준으로 소속 아파트를 조회한다.

사용자가 다른 아파트의 `apartmentId` 를 임의로 전달하여 조회하는 방식은 제공하지 않는다.

6. 공지사항 API

D-01. 공지사항 목록 조회

```
GET /api/v1/notices
```

Query Parameter

```
page
size
important
```

예시

```
GET /api/v1/notices?page=0&size=10&important=true
```

Response

```
{
  "resultCode": "200-1",
```

```

"msg": "공지사항을 조회했습니다.",
"data": {
  "content": [
    {
      "noticeId": 1,
      "title": "주차장 공사 안내",
      "isImportant": true,
      "publishedAt": "2026-08-13T10:00:00"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 15
}
}

```

핵심 정책

사용자의 소속 아파트 기준으로만 조회한다.

```

JWT
↓
apartmentId
↓
WHERE notices.apartment_id = ?

```

D-02. 공지사항 상세 조회

```
GET /api/v1/notices/{noticeId}
```

Response

```

{
  "resultCode": "200-1",
  "msg": "공지사항을 조회했습니다.",
  "data": {
    "noticeId": 1,
    "title": "주차장 공사 안내",
    "content": "2026년 8월 20일부터...",
    "isImportant": true,
    "publishedAt": "2026-08-13T10:00:00"
  }
}

```

접근 제어

공지사항의 `apartment_id`와 사용자 `apartment_id`가 일치하는 경우만 조회한다.

7. 관리자 공지사항 API

D-03. 공지사항 등록

POST /api/v1/admin/notices

Request

```
{
  "title": "주차장 공사 안내",
  "content": "2026년 8월 20일부터...",
  "isImportant": true
}
```

Response

```
{
  "resultCode": "200-1",
  "msg": "공지사항이 등록되었습니다.",
  "data": {
    "noticeId": 1
  }
}
```

아파트 ID는 Request에서 받기보다는 관리자의 관리 아파트 정보를 기준으로 결정한다.

D-04. 공지사항 수정

PATCH /api/v1/admin/notices/{noticeId}

Request

```
{
  "title": "수정된 주차장 공사 안내",
  "content": "수정된 내용입니다.",
  "isImportant": true
}
```

D-05. 공지사항 삭제

```
DELETE /api/v1/admin/notices/{noticeId}
```

관리자가 관리하는 아파트의 공지사항인지 검증한다.

8. 긴급 알림 API

E-01. 긴급 알림 목록

```
GET /api/v1/emergency-alerts
```

Response

```
{
  "resultCode": "200-1",
  "msg": "긴급 알림을 조회했습니다.",
  "data": {
    "content": [
      {
        "alertId": 1,
        "title": "103동 엘리베이터 점검",
        "location": "103동",
        "status": "ACTIVE",
        "startedAt": "2026-08-13T14:00:00"
      }
    ]
  }
}
```

E-02. 긴급 알림 상세

```
GET /api/v1/emergency-alerts/{alertId}
```

E-03. 긴급 알림 등록

```
POST /api/v1/admin/emergency-alerts
```

Request

```
{
  "title": "103동 엘리베이터 고장",
```

```
"content": "103동 엘리베이터 점검이 진행됩니다.",
"location": "103동 1호기"
}
```

등록 시 자동으로:

```
status = ACTIVE
startedAt = 현재 시간
```

E-04. 긴급 알림 종료

```
PATCH /api/v1/admin/emergency-alerts/{alertId}/resolve
```

Response

```
{
  "resultCode": "200-1",
  "msg": "긴급 알림이 종료되었습니다.",
  "data": {
    "alertId": 1,
    "status": "RESOLVED",
    "resolvedAt": "2026-08-13T16:30:00"
  }
}
```

9. 민원 API

F-01. 민원 등록

```
POST /api/v1/complaints
```

Request

```
{
  "category": "FACILITY",
  "title": "103동 계단 난간 파손",
  "content": "5층 계단 난간이 파손되어 있습니다.",
  "location": "103동 5층 계단"
}
```

Response

```
{
  "resultCode": "200-1",
  "msg": "민원이 등록되었습니다.",
  "data": {
    "complaintId": 1,
    "status": "RECEIVED"
  }
}
```

F-02. 민원 첨부파일 업로드

POST /api/v1/complaints/{complaintId}/attachments

Content-Type

multipart/form-data

Request

file: image.jpg

Response

```
{
  "resultCode": "200-1",
  "msg": "파일이 업로드되었습니다.",
  "data": {
    "attachmentId": 10
  }
}
```

파일 자체는 Object Storage에 저장하고 PostgreSQL에는 `object_key`를 저장한다.

F-03. 내 민원 목록

GET /api/v1/complaints/me

Query Parameter

status
page

size

Response

```
{
  "resultCode": "200-1",
  "msg": "민원을 조회했습니다.",
  "data": {
    "content": [
      {
        "complaintId": 1,
        "title": "103동 계단 난간 파손",
        "category": "FACILITY",
        "status": "PROCESSING",
        "createdAt": "2026-08-13T10:00:00"
      }
    ]
  }
}
```

F-04. 민원 상세

GET /api/v1/complaints/{complaintId}

본인이 등록한 민원만 조회할 수 있다.

10. 관리자 민원 API

F-05. 민원 목록

GET /api/v1/admin/complaints

Query Parameter

status
category
page
size

관리자가 관리하는 아파트의 민원만 조회한다.

F-06. 민원 상태 변경

```
PATCH /api/v1/admin/complaints/{complaintId}/status
```

Request

```
{
  "status": "PROCESSING",
  "resolution": null
}
```

처리 완료:

```
{
  "status": "COMPLETED",
  "resolution": "난간 교체를 완료했습니다."
}
```

상태 전이

```
RECEIVED
  ↓
CHECKING
  ↓
PROCESSING
  ↓
COMPLETED
```

11. 관리비 API

G-01. 내 관리비 목록

```
GET /api/v1/management-fees/me
```

Query Parameter

```
year
```

Response

```
{
  "resultCode": "200-1",
  "msg": "관리비를 조회했습니다.",
  "data": {
    "content": [
```

```
{
  "feeId": 10,
  "billingYear": 2026,
  "billingMonth": 8,
  "totalAmount": 350000
}
```

12. 관리비 상세 API

G-02. 관리비 상세

GET /api/v1/management-fees/{feeId}

Response

```
{
  "resultCode": "200-1",
  "msg": "관리비 상세 정보를 조회했습니다.",
  "data": {
    "feeId": 10,
    "billingYear": 2026,
    "billingMonth": 8,
    "totalAmount": 350000,
    "items": [
      {
        "category": "GENERAL_MANAGEMENT",
        "amount": 80000
      },
      {
        "category": "ELECTRICITY",
        "amount": 120000
      },
      {
        "category": "HEATING",
        "amount": 60000
      }
    ]
  }
}
```

13. 관리비 분석 API

G-03. 관리비 이상 분석

GET /api/v1/management-fees/{feeId}/analysis

Response

```
{
  "resultCode": "200-1",
  "msg": "관리비를 분석했습니다.",
  "data": {
    "currentTotal": 350000,
    "previousTotal": 285000,
    "changeRate": 22.8,
    "sameAreaAverage": 310000,
    "items": [
      {
        "category": "ELECTRICITY",
        "currentAmount": 120000,
        "previousAmount": 90000,
        "changeRate": 33.3,
        "difference": 30000
      },
      {
        "category": "HEATING",
        "currentAmount": 60000,
        "previousAmount": 50000,
        "changeRate": 20.0,
        "difference": 10000
      }
    ]
  }
}
```

처리 구조

```
PostgreSQL
↓
전월 관리비 조회
↓
동일 평형 평균 계산
↓
항목별 증감 계산
↓
분석 결과 생성
```


↓
Frontend

AI를 이용한 자연어 원인 분석을 추가할 경우 별도의 AI API로 확장할 수 있다.

14. 관리규정 RAG API

RAG는 일반 Spring Boot API와 별도의 AWS Lambda 시스템으로 구현한다.

```
Client
↓
API Gateway
↓
Lambda
├─ OpenSearch
└─ Bedrock
```

15. RAG 채팅 API

R-01. 관리규정 질문

POST /rag/v1/chat

인증

JWT 필요

Request

```
{
  "message": "주차 등록 차량은 몇 대까지 가능한가요?"
}
```

아파트를 명시하지 않은 경우 사용자의 JWT에 포함된 소속 아파트를 기준으로 검색한다.

특정 아파트 질문

```
{
  "message": "00아파트 주차 등록 차량은 몇 대까지 가능한가요?"
}
```

질문에서 아파트를 식별하여 해당 아파트의 공개 규정을 검색한다.

Response

```
{
  "resultCode": "200-1",
  "msg": "답변을 생성했습니다.",
  "data": {
    "answer": "주차 등록은 세대당 차량 2대까지 가능합니다.",
    "sources": [
      {
        "apartmentName": "00아파트",
        "documentName": "주차관리규정",
        "article": "제5조",
        "page": 3
      }
    ]
  }
}
```

16. RAG 검색 정책

RAG API는 질문에 따라 검색 범위를 다르게 적용한다.

Case 1. 아파트를 지정하지 않음

```
"주차 규정이 어떻게 돼?"
↓
JWT
↓
사용자 apartment_id
↓
해당 아파트 규정 검색
```

Case 2. 다른 아파트 지정

```
"00아파트 주차 규정은?"
↓
00아파트 식별
↓
apartment_id = 00아파트
↓
is_public = true
↓
검색
```

Case 3. 비교 질문

```
"우리 아파트와 00아파트의 주차 규정을 비교해줘."
↓
우리 아파트
+
```

00아파트

↓

각각 검색

↓

Bedrock

↓

비교 결과

중요 정책

내 아파트

→ 공개/비공개 규정 모두 검색 가능

다른 아파트

→ is_public = true인 규정만 검색 가능

17. 관리자 RAG 문서 등록 API

R-02. 관리규정 등록

POST /rag/v1/admin/documents

인증

ADMIN

Request

multipart/form-data

file: management-regulation.pdf
documentType: MANAGEMENT_REGULATION
apartmentId: 1
isPublic: true

처리 과정

PDF Upload

↓

S3

↓

Lambda

↓

텍스트 추출

↓

Chunking

↓

Bedrock Embedding

↓
OpenSearch Index

Response

```
{
  "resultCode": "200-1",
  "msg": "관리규정이 등록되었습니다.",
  "data": {
    "documentId": 10,
    "status": "PROCESSING"
  }
}
```

문서 임베딩은 시간이 걸릴 수 있으므로 비동기 처리한다.

18. 관리자 RAG 문서 목록

R-03. 관리규정 목록

GET /rag/v1/admin/documents

Query Parameter

apartmentId
documentType
page
size

Response

```
{
  "resultCode": "200-1",
  "msg": "관리규정 목록을 조회했습니다.",
  "data": {
    "content": [
      {
        "documentId": 10,
        "apartmentId": 1,
        "documentName": "주차관리규정",
        "documentType": "PARKING_REGULATION",
        "version": "2026.01",
        "isPublic": true
      }
    ]
  }
}
```

```
}  
}
```

19. 관리자 RAG 문서 삭제

R-04. 관리규정 삭제

```
DELETE /rag/v1/admin/documents/{documentId}
```

처리

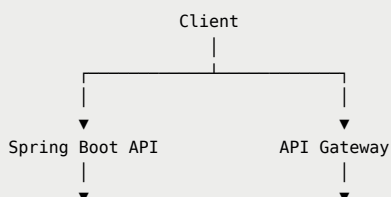
문서 삭제 요청
↓
S3 파일 삭제
↓
OpenSearch 관련 Vector 삭제
↓
문서 메타데이터 삭제/비활성화

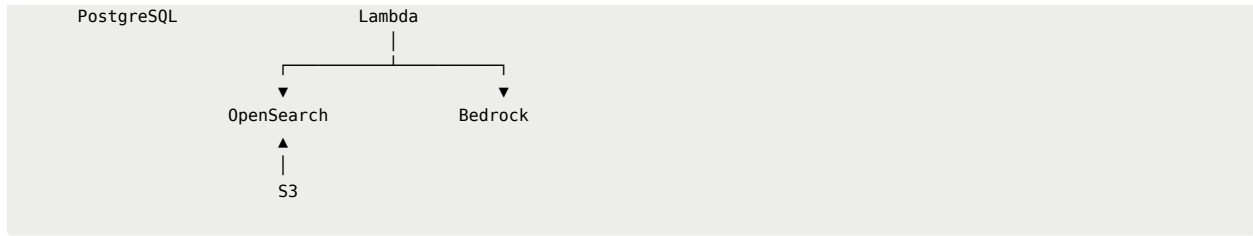
20. API별 데이터 접근 정책

API	사용자 데이터 범위
공지사항	본인 소속 아파트
긴급 알림	본인 소속 아파트
민원	본인이 등록한 민원
관리자 민원	관리 대상 아파트
관리비	본인 세대
관리비 평균	비식별 집계 데이터
RAG 내 아파트 질문	본인 소속 아파트
RAG 다른 아파트 질문	<code>is_public=true</code> 규정
관리자 RAG	관리 대상 아파트

21. API 시스템 분리

최종적으로 API는 크게 두 영역으로 나눈다.





Spring Boot API

담당:

- 회원가입 / 로그인
- JWT 인증
- 아파트 / 세대
- 공지사항
- 긴급 알림
- 민원
- 관리비
- 관리자 기능

RAG API

담당:

- 관리규정 질문
- 규정 검색
- 다른 아파트 공개 규정 검색
- 문서 업로드
- 문서 임베딩
- Vector Search
- 근거 기반 AI 답변